Coroutine Implementation

CSE 493L

Coroutine API

- yield
  - called from within a coroutine to suspend execution
- spawn
  - create a new coroutine
- join
  - wait for a coroutine to finish

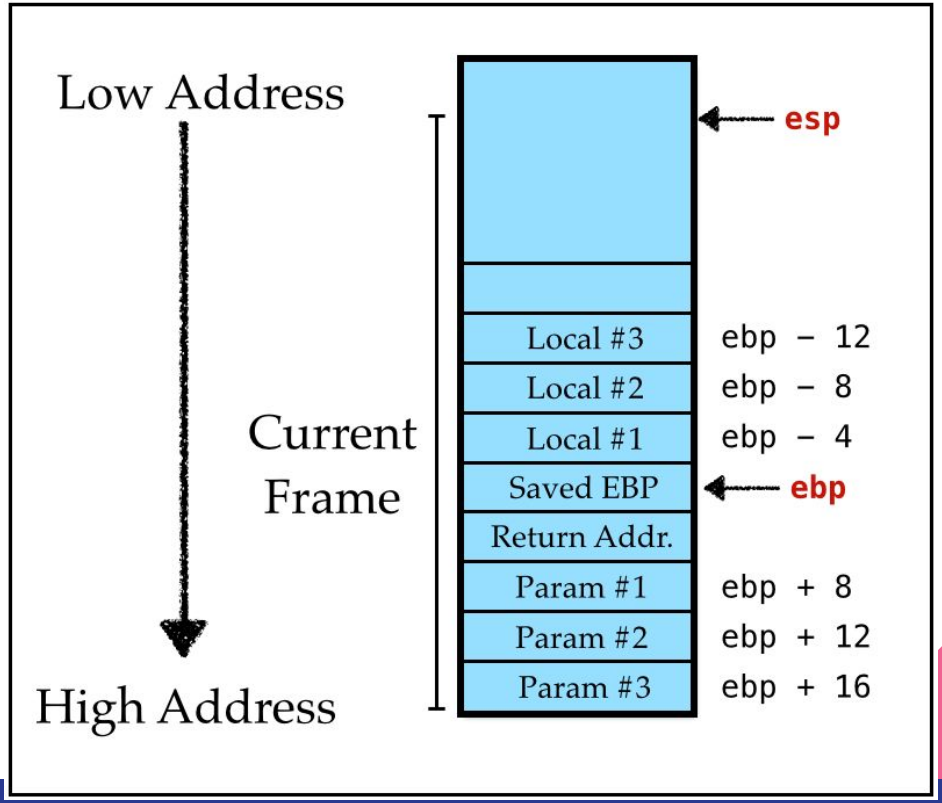
How do we spawn a new coroutine?

What does any routine need to run?

A stack!

How do we create a stack? Just malloc() some memory for it!

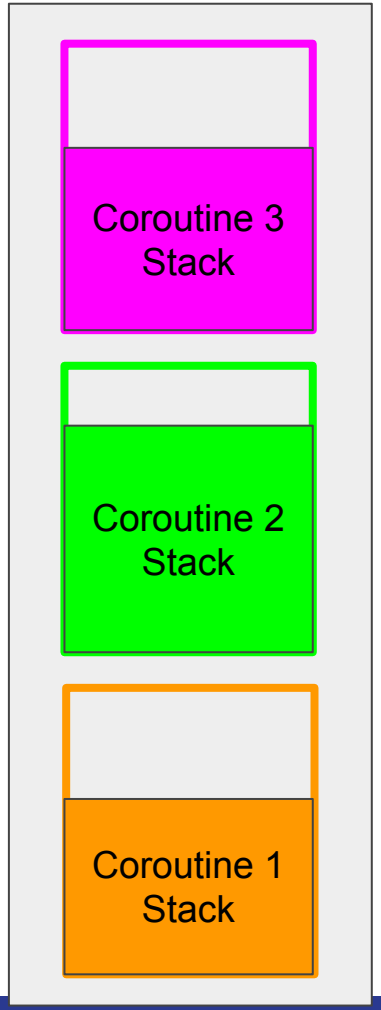
A quick recap: x86 stack layout



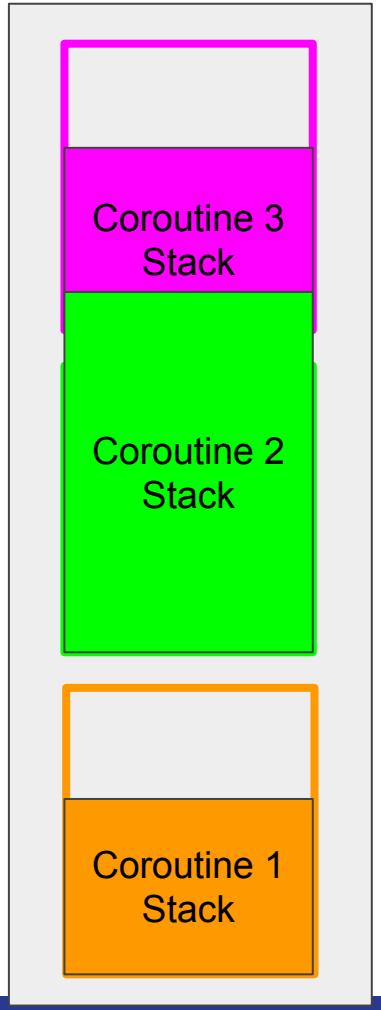
Potential Problem

If we malloc a stack and set %rsp to point to the bottom of it, what could go wrong?

Coroutine 2 is growing…



Stack Overflow!



How do you protect from stack overflows?

1. Ask the OS to “protect” a chunk of memory from reads/writes and to notify you when it happens
2. Assume they don’t happen
    a. this is our approach for homework 1

Let’s talk about yield

It must switch to a new coroutine

How do we do this!?

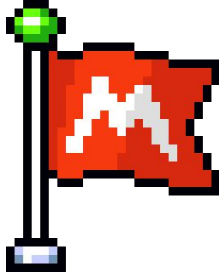
Context Switch

1. Figure out next coroutine to run
2. Checkpoint all of your state
3. Load next coroutine’s previous checkpoint

1. Figure out next coroutine to run

- List of ready coroutines
  - Pop one at random, or in FIFO order
  - “The Scheduler”

2. Checkpoint all of your state



What is a routine's state? Discuss with a partner.

A program has:

- Memory
  - Stack
  - Heap
  - Read-only / Text / Code
- Registers
  - General Purpose Registers (rax, rdi, rsp, … on x86_64)
  - Program Counter (rip on x86_64)

Let’s make an assumption

Coroutines do not touch the memory of other coroutines

Corollary: We don’t have to checkpoint or restore memory

2. Checkpoint all of your state

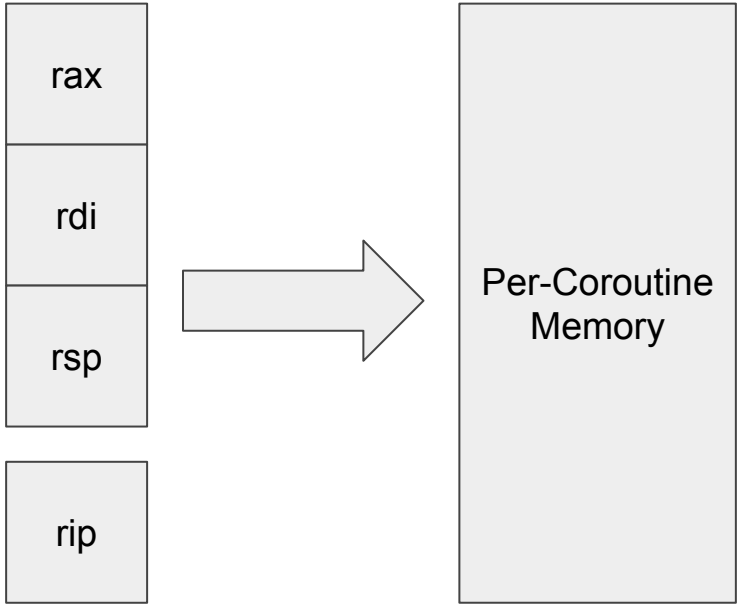
A program has:

- Memory
  - Stack
  - Heap
  - Read-only / Text / Code
- Registers
  - General Purpose Registers (rax, rdi, rsp, … on x86_64)
  - Program Counter (rip on x86_64)

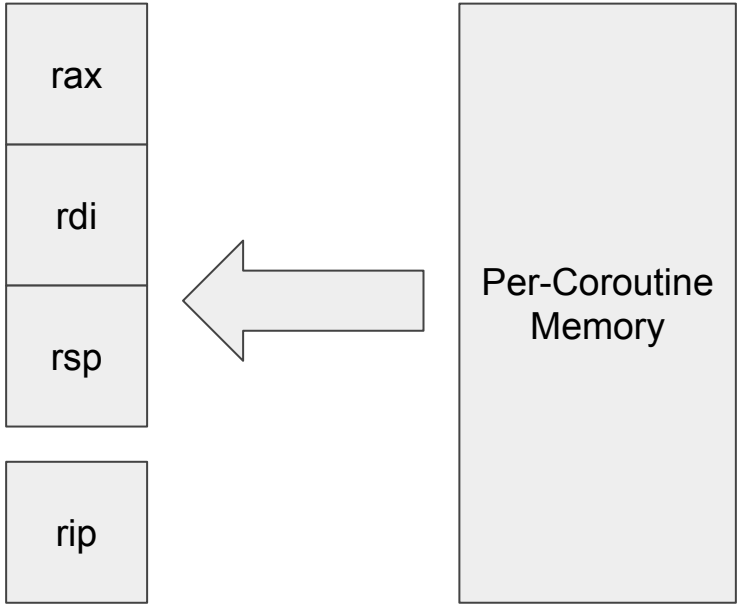
No need to checkpoint!

Do we need to checkpoint?

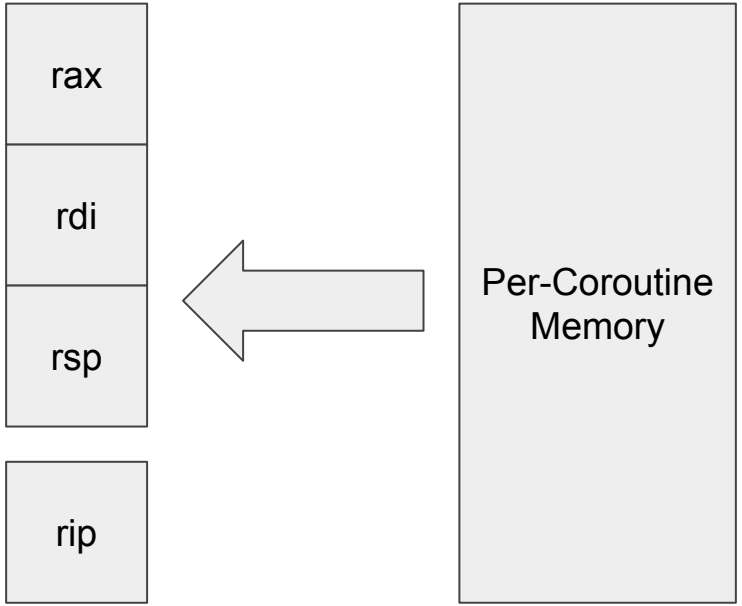
2. Checkpoint all of your state



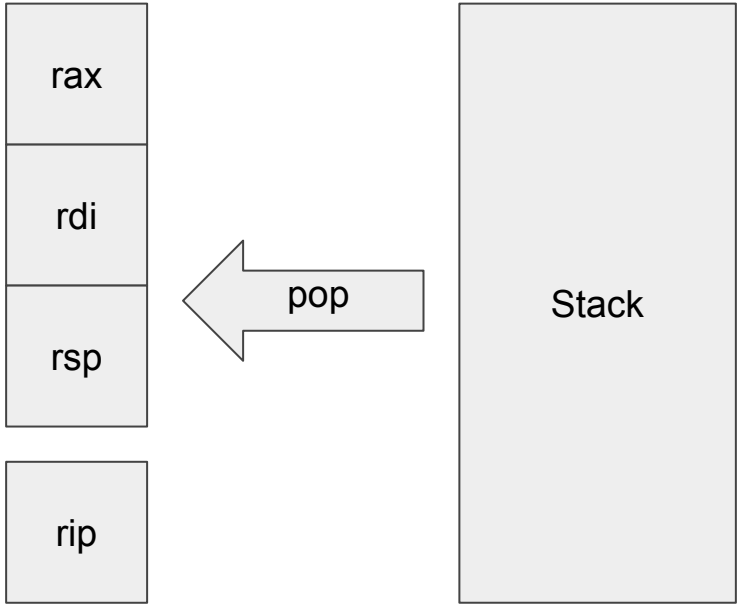
3. Load next coroutine’s previous checkpoint



What is some convenient per-coroutine memory we have?



What is some convenient per-coroutine memory we have?



Let’s revise the process

1. Pick a next coroutine
2. Push rip to stack
3. Push rax, rdi, … to stack
4. Store stack pointer (rsp) in prev coroutine struct
5. Load new stack pointer (rsp) from next coroutine struct
6. Pop rax, rdi, …
7. Pop rip

Pseudocode

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

Assembly Function

```
// void switch(reg* next_rsp, reg* prev_rsp);

switch:
    push %r12          mov %rsp, (%rsi)          pop %rbp
    push %r13          mov (%rdi), %rsp          pop %rbx
    push %r14                                    pop %r15
    push %r15                                    pop %r14
    push %rbx                                    pop %r13
    push %rbp                                    pop %r12

                                                 ret
```

x86_64 Calling Convention

Register	Callee Save	Description
%rax		result register; also used in idiv and imul instructions.
%rbx	yes	miscellaneous register
%rcx		fourth argument register
%rdx		third argument register; also used in idiv and imul instructions.
%rsp		stack pointer
%rbp	yes	frame pointer
%rsi		second argument register
%rdi		first argument register
%r8		fifth argument register
%r9		sixth argument register
%r10		miscellaneous register
%r11		miscellaneous register
%r12-%r15	yes	miscellaneous registers

Pseudocode

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

The compiler knows to store the caller-saved registers before calling switch!

```
// void switch(reg* next_rsp, reg* prev_rsp);

switch:
    push %r12
    push %r13
    push %r14
    push %r15
    push %rbx
    push %rbp

    mov %rsp, (%rsi)
    mov (%rdi), %rsp

    pop %rbp
    pop %rbx
    pop %r15
    pop %r14
    pop %r13
    pop %r12

    ret
```

But wait! How do we store %rip?

How do we store RIP?

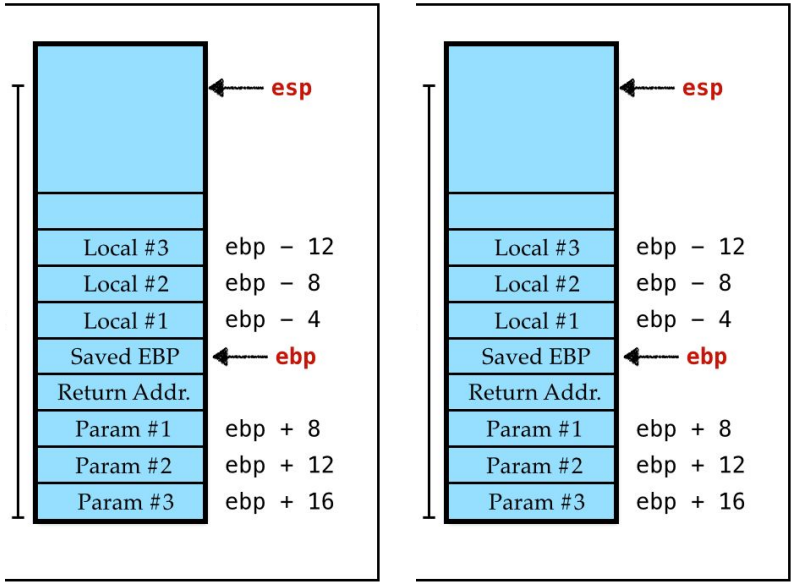
```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

This `call` to switch() places RIP on the stack!

How do we store RIP?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

The **ret** in switch will jump back to the appropriate coroutine after switching stacks!



Context Switch

And poof, we’re in a different coroutine!

Coroutine Struct

Needs its own stack pointer and stack itself

Needs an associated channel

```
struct Coroutine {
    uint64_t rsp;
    uint8_t my_stack[STACK_SIZE];
    Channel *ch;
}
```

Channels

We need to implement blocking with channels

What is blocking?

Blocking

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

Channel

```
struct Channel {
    CoroutineQueue blockedSenders;
    CoroutineQueue blockedReceivers;
}
```

send(ch, data)

```
if ch.blockedReceivers.empty() {
    yieldOnto(ch.blockedSenders)
    // now that we're back, ???
} else {
    r = ch.blockedReceivers.pop();
    // somehow get data to r
    ready_queue.push(r)
}
```

Food for thought

The fundamental assumption that we made: one coroutine will not touch another coroutine's memory. When is it safe to:

- Send / receive pointers to local data?
- Send / receive pointers to malloc'd data?
- Use global variables?

Good luck with Homework 1!

Any questions?